# 01 · Was ist Echtzeit

Zeitvertrag, Polling und Interrupts · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

**Folie** 1 / 50

## 01 · Was ist Echtzeit?

- Ein richtiges Ergebnis braucht den richtigen Zeitpunkt
- Grundlagen, Polling und Interrupts
- Theorie und ein kleiner Versuch auf dem NUCLEO-H755ZI-Q
- Prof. Dr.-Ing. Daniel Klünder · DHBW Stuttgart

**Notizen** 1 / 50

Willkommen zur Vorlesung Echtzeitsysteme, kurz EZS, an der Dualen Hochschule Baden-Württemberg, kurz DHBW, in Stuttgart. Ein Echtzeitsystem muss eine geforderte Funktion innerhalb einer vorgegebenen Zeit erfüllen. Die Bezeichnung sagt zunächst nichts über eine besonders hohe Rechengeschwindigkeit aus. Heute lernen Sie, eine zeitliche Anforderung zu formulieren und erste Programmstrukturen daran zu beurteilen.
Das NUCLEO-H755ZI-Q ist das Entwicklungsboard des Kurses. Es enthält einen Mikrocontroller, also einen Baustein mit Prozessorkernen, Speicher und Ein-/Ausgabefunktionen. Der Versuch dient der Anschauung; die Theorie lässt sich ohne Board nachvollziehen. Präsentationsfolien enthalten die Kernaussagen, diese Notizen den zusammenhängenden Lerntext. Die Papieraufgaben und das Quiz prüfen dieselben Unterscheidungen. Ein vorhandenes Echtzeitbetriebssystem wird für den Einstieg weder vorausgesetzt noch benötigt.

**Folie** 2 / 50

## Sie beginnen ohne Echtzeit-Vorkenntnisse

- Vorausgesetzt: Funktionen, Schleifen und Variablen in C
- Grundidee eines digitalen Ein- oder Ausgangs
- Neue Begriffe werden an einem Beispiel eingeführt
- Betriebssysteme und mehrere Kerne folgen schrittweise

**Notizen** 2 / 50

Die fachliche Grundlage dieses ersten Termins ist elementares Programmieren: Eine Funktion erledigt einen Arbeitsschritt und kehrt zum Aufrufer zurück. Eine Schleife wiederholt Arbeit; eine Variable speichert einen Wert. C ist die Programmiersprache der späteren Anwendungsbausteine. Für das Verständnis genügt zunächst, den Kontrollfluss lesen zu können. Ein digitaler Eingang nimmt zwei logische Pegel an; ein Ausgang kann zwischen ihnen umschalten.
Sie müssen noch nicht wissen, wie ein Betriebssystem Arbeit auswählt, wie ein Interrupt intern Register sichert oder wie zwei Kerne Daten austauschen. Wir erklären nur die Mechanismen, die für die heutige Fragestellung nötig sind. Die Übung beginnt deshalb mit Zeitverträgen und Rechenaufgaben auf Papier. Der Programmierteil setzt eine vorbereitete Werkzeuginstallation voraus. Fehlende Installationserfahrung ist keine fehlende Echtzeitkompetenz und wird getrennt vom fachlichen Lernziel behandelt. Eine kurze Wiederholung von Zeigern erfolgt dort, wo der Timer-Callback sie erstmals benutzt.

**Folie** 3 / 50

## Was Sie danach begründen können

1. Funktion und Rechtzeitigkeit getrennt beurteilen
2. Harte, feste und weiche Anforderungen unterscheiden
3. Antwortzeit, Rechenbedarf, Latenz und Jitter berechnen
4. Polling und Interrupts mit ihren Grenzen vergleichen
5. Beobachtungen von einer Zeitgarantie unterscheiden

**Notizen** 3 / 50

Diese Lernziele sind konkrete Tätigkeiten. Sie sollen einen Ereigniszeitpunkt und den geforderten Abschluss benennen, eine Frist berechnen und erklären, weshalb ein korrekter Zahlenwert trotzdem zu spät sein kann. Harte, feste und weiche Echtzeit beschreiben unterschiedliche Verträge über Fristverletzungen und den Nutzen verspäteter Ergebnisse. Die Begriffe werden gleich eingeführt, nicht als bekannte Kategorien vorausgesetzt.
Antwortzeit misst die Dauer vom vereinbarten Beginn bis zum Abschluss. Rechenbedarf zählt dagegen nur die eigene Prozessorarbeit. Latenz verlangt ebenfalls zwei benannte Endpunkte; Jitter beschreibt die Schwankung einer solchen Größe über mehrere Ausführungen. Polling bedeutet wiederholtes Abfragen, ein Interrupt eine Unterbrechungsanforderung an den Prozessor. Beide können passende Aufgaben lösen, beide benötigen Zeitannahmen. Am Ende sollen Sie nicht nur „die LED blinkt“ berichten, sondern sagen, welche Größe beobachtet wurde und welche Aussage daraus folgt. LED steht für Light-Emitting Diode, eine Leuchtdiode.

**Folie** 4 / 50

## Vier Schulstunden mit Theorie und Praxis

| Tätigkeit | Minuten |
|---|---|
| Anforderungen und Zeitgrößen · Folien 1–24 | 45 |
| Programmstrukturen und Versuch · Folien 25–50 | 45 |
| Vier Papieraufgaben mit Besprechung | 40 |
| Vorbereiteter STM32-Versuch | 35 |
| Quiz und gemeinsame Klärung | 15 |
| Gesamt | 180 |

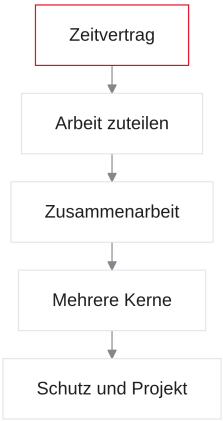
**Notizen** 4 / 50

Vier Schulstunden mit jeweils 45 Minuten ergeben 180 Minuten. Die beiden Vortragsblöcke enthalten kurze Verständnisfragen; die etwa 50 Folien sind keine Verpflichtung, jede Folie gleich lange zu besprechen. Die erste Hälfte baut den Zeitvertrag auf, die zweite verbindet ihn mit der Programmstruktur. Die Papieraufgaben üben die Art von Begründungen und Rechnungen, die auch in einer Klausur vorkommen können.

STM32 bezeichnet die Mikrocontrollerfamilie des Laborboards. Für die 35 Minuten Praxis müssen das Konfigurationswerkzeug CubeMX, Visual Studio Code (VS Code) als Editor mit der STM32-Erweiterung, die Übersetzungswerkzeuge und der Boardzugang bereits funktionieren. Eine Erstinstallation und das erstmalige Einrichten eines Dual-Core-Projekts gehören in eine Vorbereitung oder betreute Zusatzzeit. Ohne Board können alle vier Papieraufgaben bearbeitet und besprochen werden. Der Praxisblock dient dann dem Lesen des vollständigen Anwendungsmoduls und der Planung eines Versuchs; daraus entstehen ausdrücklich keine Hardwaremesswerte. Die gemeinsame Quizbesprechung klärt insbesondere naheliegende Begriffsverwechslungen.

**Folie** 5 / 50

## Der rote Faden des Kurses



**Notizen** 5 / 50

Die Abbildung wird von oben nach unten gelesen. Ein Zeitvertrag legt fest, welche Reaktion bis wann erforderlich ist. Erst danach ist die Frage sinnvoll, wie mehrere Arbeiten Prozessorzeit erhalten. Das Betriebssystem kann die Verwaltung solcher Arbeiten übernehmen; die Zusammenarbeit betrifft gemeinsame Daten und die Übergabe von Informationen. Mehrere Kerne vergrößern die Gestaltungsmöglichkeiten, bringen aber gemeinsame Ressourcen und zusätzliche Annahmen mit. Beobachtung macht tatsächliche Abläufe sichtbar. Schutzmechanismen behandeln bestimmte Fehler; das Abschlussprojekt verbindet die Bausteine.
Die Pfeile bedeuten einen didaktischen Aufbau, keine Zeitachsen und keine automatisch erfüllten Garantien. Der rot umrandete Startpunkt markiert das Thema dieses Termins. Heute arbeiten wir mit einem einzigen Anwendungskern und einfachen Ablaufmustern. Die nächste Vorlesung beschäftigt sich mit Scheduling, der Zuteilung von Prozessorzeit. Erst später führen wir ein Echtzeitbetriebssystem ein. Diese Reihenfolge verhindert, dass ein Werkzeugname an die Stelle eines begründeten Zeitvertrags tritt.

**Folie** 6 / 50

## Ein durchgängiges Beispiel: Sortierklappe

- Ein Sensor meldet ein Werkstück
- Die Software entscheidet über dessen Weg
- Ein Ausgang gibt den Stellauftrag an die Klappe
- Lehrannahme: Abschluss innerhalb von $10$ ms

Die Frist und die vereinfachte Strecke sind ein Lehrmodell.

**Notizen** 6 / 50

Stellen Sie sich ein Förderband mit einer Sortierklappe vor. Ein Sensor erkennt ein Werkstück; die Software berechnet einen Stellauftrag und gibt ihn an einen Ausgang. Für das Lehrmodell verlangen wir, dass der richtige Auftrag spätestens 10 Millisekunden nach dem vereinbarten Sensorereignis am Ausgang vorliegt. Eine Millisekunde ist ein Tausendstel einer Sekunde. Die Zahl ist keine Spezifikation einer realen Maschine.
Der Abschluss dieses Softwarevertrags ist die Ausgabe des Stellauftrags. Die anschließende mechanische Bewegung der Klappe ist nicht darin enthalten. Würde die Anforderung stattdessen „Klappe vollständig in Position“ lauten, müssten Aktordauer und Mechanik ergänzt werden. Genau diese Abgrenzung verhindert, dass eine schnelle Berechnung mit einer schnellen physischen Reaktion verwechselt wird. Zunächst nehmen wir an, dass das Ereignis zuverlässig erfasst wird und seine Freigabe sofort erfolgt. Später betrachten wir, weshalb schon diese Annahme bei kurzen Eingangspulsen fehlen kann. Nun können wir Wert und Zeitpunkt des Auftrags getrennt beurteilen.

**Folie** 7 / 50

## Richtig und rechtzeitig sind zwei Prüfungen

| Stellauftrag | Vor oder auf der Frist | Nach der Frist |
|---|---|---|
| Fachlich richtig | Vertrag erfüllt | Zeitbedingung verletzt |
| Fachlich falsch | Funktion verletzt | Beide Bedingungen verletzt |

Echtzeitkorrektheit verlangt Funktion und Rechtzeitigkeit.

**Notizen** 7 / 50

Funktionale Korrektheit bedeutet, dass der ausgegebene Stellauftrag zur verlangten Sortierentscheidung passt. Rechtzeitigkeit bedeutet, dass sein definierter Abschluss die Frist einhält. Die Tabelle kombiniert beide Prüfungen. Nur oben links sind beide Bedingungen erfüllt. Ein früh ausgegebener falscher Stellauftrag bleibt falsch; ein später richtiger Auftrag kann das Werkstück bereits verfehlt haben.
Wir zählen die Fertigstellung genau auf der Deadline, also auf dem Fristzeitpunkt, als rechtzeitig. Das ist eine ausdrücklich gewählte Konvention; ein realer Vertrag kann strengere Bedingungen einschließlich einer Reserve verlangen. Bei Sensorereignis 2 Millisekunden und zulässiger Antwortdauer 10 Millisekunden liegt die Deadline bei 12 Millisekunden. Ein richtiger Auftrag bei 12 erfüllt die Modellfrist, bei 12,1 verletzt er sie. Ob ein verspäteter Auftrag noch Restnutzen hat und welche Fehlerfolge zulässig ist, muss zusätzlich spezifiziert werden. Diese Folgen begründen die späteren Echtzeitklassen und folgen nicht allein aus dem Zahlenwert der Frist.

**Folie** 8 / 50

## Aus „schnell“ wird ein Zeitvertrag

| Bestandteil | Sortierklappen-Modell |
|---|---|
| Beginn | Freigabe durch erfasstes Sensorereignis |
| Abschluss | Richtiger Stellauftrag am Ausgang |
| Relative Frist | $D = 10\,\text{ms}$ |
| Bedingung | $f \leq r + D$ |

$r$: Freigabezeitpunkt · $f$: Fertigstellungszeitpunkt

**Notizen** 8 / 50

Ein prüfbarer Zeitvertrag nennt Beginn, Abschluss, zulässige Dauer und relevante Randbedingungen. Der Freigabezeitpunkt $r$, englisch release, ist hier der Moment, ab dem die Software den erfassten Auftrag bearbeiten darf. Der Fertigstellungszeitpunkt $f$, englisch finish oder completion, liegt bei der geforderten

**Notizen, Fortsetzung** 8 / 50

Ausgabe. Die relative Deadline $D$ ist eine Dauer ab Freigabe. Die absolute Deadline $d = r + D$ ist dagegen ein Zeitpunkt auf derselben Uhr.
Die Bedingung $f \leq r + D$ bedeutet: Der Abschluss darf spätestens auf der absoluten Deadline liegen. Alle Zeitpunkte und Dauern müssen auf einen gemeinsamen Zeitbezug und dieselbe Einheit gebracht werden. Bei $r = 2$ ms und $D = 10$ ms ergibt sich $d = 12$ ms. „Sehr schnell“ benennt weder dieses Bezugsereignis noch einen zulässigen Maximalwert. Für einen vollständigen Vertrag fehlen außerdem Lastannahmen und die Behandlung einer Fristverletzung. Wir halten im Lehrmodell zunächst einen Auftrag und die genannten Endpunkte fest und ergänzen nun die Fehlerfolge.

**Folie** 9 / 50

## Drei Arten von Echtzeitanforderungen

| Klasse | Vertrag über eine Fristverletzung |
|---|---|
| Hart · Hard | Kein Miss ist zulässig |
| Fest · Firm | Spätes Einzelergebnis wertlos, begrenzte Misses können zulässig sein |
| Weich · Soft | Spätes Ergebnis kann Restnutzen haben |

Die Klasse gehört zur Aufgabe und ihrem Vertrag.

**Notizen** 9 / 50

Ein Miss, englisch deadline miss, ist eine Verletzung der vereinbarten Frist. Harte Echtzeit, Hard Real-Time, verlangt unter den dokumentierten Betriebsannahmen die Einhaltung jeder betreffenden Frist. Ein beobachteter Miss widerlegt dann den behaupteten Vertrag für diesen Fall. Harte Echtzeit bedeutet nicht automatisch, dass jede Verletzung eine Katastrophe auslöst; die Sicherheitsfolge ist eine weitere Anforderung.
Bei fester Echtzeit, Firm Real-Time, ist ein verspätetes Einzelergebnis für seinen vorgesehenen Zweck wertlos. Ein Systemvertrag kann zugleich eine begrenzte Zahl oder Rate von Auslassungen gestatten. Diese Grenze muss benannt sein, statt „gelegentlich“ zu sagen. Bei weicher Echtzeit, Soft Real-Time, kann ein später Abschluss noch Nutzen haben, typischerweise mit verminderter Qualität. Ein später Filmrahmen kann beispielsweise weniger nützlich sein, ohne dass deshalb jede Medienaufgabe automatisch weich wäre. Wir orientieren diese Begriffe an der Kursliteratur von Buttazzo. Die Klasse entsteht aus der konkreten Anforderung, nicht aus Branche, Prozessortyp oder Betriebssystemname.

**Folie** 10 / 50

## Harte Echtzeit braucht benannte Annahmen

- Sortierauftrag: jeder Abschluss innerhalb von $10$ ms
- Zulässige Eingangsrate und andere Lasten festlegen
- Jeder Miss verletzt diesen Vertrag
- Fehlerreaktion zusätzlich spezifizieren

Ein schneller Durchschnitt ersetzt keine obere Grenze.

**Notizen** 10 / 50

Für die Sortierklappe nehmen wir als harten Vertrag an: Jeder angenommene Auftrag wird fachlich korrekt innerhalb von 10 Millisekunden abgeschlossen. „Jeder“ bezieht sich auf einen festgelegten Betriebsbereich. Dazu gehören etwa die kleinste Zeit zwischen zwei Werkstücken, zulässige andere Arbeiten und die gewählte Hardwarekonfiguration. Eine beliebig hohe Ereignisrate kann kein endlicher Prozessor bewältigen.
Ein Miss ist eine Fristverletzung und in diesem Betriebsbereich nicht zulässig. Die Frage, was bei Überlast, Sensorausfall oder einem Miss geschehen soll, ist trotzdem gesondert zu beantworten. Denkbar wäre ein kontrolliertes Anhalten; eine solche Sicherheitsfunktion wird im heutigen LED-Versuch nicht implementiert. Der Begriff harte Echtzeit ist deshalb weder ein Gütesiegel für alle denkbaren Fehler noch gleichbedeutend mit besonders schnellem Rechnen. Ein Mittelwert unter 10 Millisekunden kann einzelne spätere Antworten verdecken. Für die Garantie brauchen wir eine begründete obere Antwortzeitschranke unter den genannten Annahmen. Die folgenden Klassen ändern den Vertrag über verspätete Ergebnisse, nicht die Bedeutung der Frist selbst.

**Folie** 11 / 50

## Feste Echtzeit: ein verspäteter Wert verfällt

Lehrvertrag für ein Prüfbild:

- Bild muss vor dem nächsten Verarbeitungsschritt vorliegen
- Späteres Bild wird verworfen
- Höchstens ein fehlendes Bild je 100 Bilder erlaubt
- Mehr Auslassungen verletzen den Gesamtvertrag

**Notizen** 11 / 50

Ein Prüfbild wird für einen bestimmten Verarbeitungsschritt benötigt. Ist dieser Schritt vorbei, darf das alte Bild nicht nachträglich als aktuelles Ergebnis benutzt werden. Sein Nutzen für diesen Zweck ist dann null. Das ist die zentrale Eigenschaft fester Echtzeit, Firm Real-Time. Wir wählen als Lehrannahme zusätzlich höchstens ein fehlendes Bild je Block von 100 Bildern; diese Zahl ist keine industrielle Kameraspezifikation. Die zulässige Auslassungsgrenze ist Teil des Gesamtvertrags. Zwei verpasste Bilder innerhalb eines solchen Blocks verletzen ihn, auch wenn alle übrigen Bilder schnell verarbeitet wurden. Ein Algorithmus

**Notizen, Fortsetzung** 11 / 50

darf aus der erlaubten Auslassung nicht ableiten, dass beliebig viele Fristverletzungen unproblematisch seien. Umgekehrt wird das verspätete Einzelergebnis nicht dadurch wieder nützlich, dass die zulässige Auslassungsrate eingehalten ist. Wertverfall und Auslassungstoleranz sind verschiedene Aussagen. Bei einer weichen Anforderung kann dagegen ein später Wert noch einen Beitrag leisten; dafür muss beschrieben werden, wie sein Nutzen mit der Verspätung zusammenhängt.

**Folie** 12 / 50

## Weiche Echtzeit: Qualität nimmt ab

Lehrvertrag für eine Statusanzeige:

- Aktualisierung möglichst innerhalb von $100$ ms
- Späterer Stand kann noch informativ sein
- Wartezeit und Auslassungen beeinflussen die Qualität
- Zulässige Qualitätseinbußen bleiben messbar festzulegen

**Notizen** 12 / 50

Eine Statusanzeige kann einen verspäteten Stand noch sinnvoll zeigen, wenn dieser für die Benutzerentscheidung weiterhin relevant ist. Die weiche Echtzeitanforderung, Soft Real-Time, beschreibt diesen verbleibenden Nutzen. Die 100 Millisekunden sind eine didaktische Zielgröße, keine allgemeine Grenze für Anzeigen. Sobald eine Anzeige eine verbindliche Warnfunktion übernimmt, kann ein anderer, möglicherweise harter Zeitvertrag nötig sein.
Weich heißt nicht „Zeit ist egal“. Ein Vertrag kann beispielsweise eine Verteilung der Aktualisierungszeiten, eine zulässige Zahl von Auslassungen oder eine maximal erlaubte Veraltung benennen. Der Restnutzen hängt außerdem von der Aufgabe ab: Ein zwei Sekunden alter Temperaturstand kann informativ sein, ein ebenso alter Kollisionshinweis bereits unbrauchbar. Daher ordnen wir einzelne Aufgaben ein. Ein und dasselbe Gerät kann eine harte Regelung, eine feste Bildverarbeitung und eine weiche Statusanzeige enthalten. Diese Kombination erklärt, weshalb eine pauschale Zuordnung anhand der Branche oder eines Gerätenamens fachlich zu grob ist.

**Folie** 13 / 50

### Verständnisfrage: drei Aufgaben, ein Gerät

| Aufgabe | Vorgegebener Vertrag |
|---|---|
| Stellauftrag | Kein Miss erlaubt |
| Prüfbild | Spät wertlos, ein Miss je 100 erlaubt |
| Anzeige | Später Stand hat Restnutzen |

Ordnen Sie die drei Klassen zu und begründen Sie Ihre Wahl.

**Notizen** 13 / 50

Die Musterlösung folgt den ausdrücklich gegebenen Verträgen: Der Stellauftrag ist hart echtzeitkritisch, weil kein deadline miss, also keine Fristverletzung, erlaubt ist. Das Prüfbild ist fest echtzeitkritisch: Ein spätes Bild ist für den konkreten Schritt wertlos, zugleich ist eine benannte Auslassung zulässig. Die Anzeige ist im Modell weich echtzeitkritisch, weil der verspätete Stand noch Restnutzen hat.
Die Aufgabe lässt sich ohne Vermutungen über eine reale Maschine lösen. Häufige Fehler sind „Industrie ist immer hart“ oder „ein Bild ist immer weich“. Beide Antworten ersetzen den Zeitvertrag durch eine Schublade. Ein weiterer Fehler besteht darin, feste Echtzeit wegen des Wertverfalls mit harter Echtzeit gleichzusetzen. Die erlaubte Auslassung unterscheidet die hier gegebenen Verträge. Nutzen nach der Frist und zulässige Verletzungsrate müssen deshalb getrennt gelesen werden. Diese Denkweise bleibt auch später wichtig: Ein schneller oder redundanter Rechner bestimmt nicht allein, welche Verletzungen das System tolerieren darf.

**Folie** 14 / 50

### Schneller im Mittel, trotzdem zu spät

Lehrwerte für vier Antwortzeiten bei $D = 10$ ms:

| System | Antworten in ms | Mittel | Größter Wert |
|---|---|---|---|
| A | 1, 1, 1, 17 | 5 | 17 |
| B | 6, 6, 6, 6 | 6 | 6 |

A verletzt eine Frist. Vier Erfolge von B beweisen keine Garantie.

**Notizen** 14 / 50

Die Antwortzeit ist hier die Dauer von der Freigabe bis zum geforderten Abschluss. Alle Zahlen sind Lehrwerte. Für System A ergibt sich der Mittelwert $(1 + 1 + 1 + 17)/4 = 5$ Millisekunden. System B benötigt im Mittel $(6 + 6 + 6 + 6)/4 = 6$ Millisekunden. A ist im Durchschnitt schneller, hat aber einen

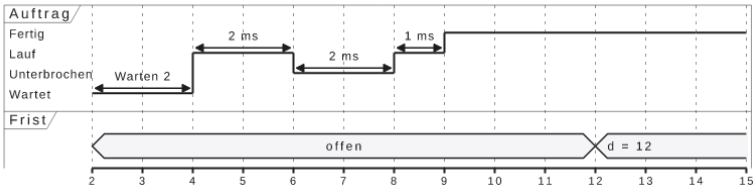
**Notizen, Fortsetzung** 14 / 50

Abschluss nach der relativen Deadline $D = 10$ Millisekunden. Sein größter Wert ist 17 und verletzt die Modellfrist um 7 Millisekunden.
Die vier beobachteten Antworten von B liegen innerhalb der Frist. Sie reichen dennoch nicht als Nachweis für alle zukünftigen Antworten unter jeder zugelassenen Last. Ein ungesehener Fall kann länger dauern. Durchsatz, also erledigte Arbeit pro Zeit, und mittlere Antwortzeit sind für andere Ziele nützlich, beantworten aber die harte Fristfrage nicht allein. Das Beispiel motiviert die präzise Zeitachse: Wir müssen wissen, welche Abschnitte in der gemessenen oder analysierten Antwortzeit enthalten sind. Andernfalls werden die Zahlen verschiedener Systeme womöglich an unterschiedlichen Endpunkten verglichen.

**Folie** 15 / 50

## Eine Ausführung auf der Zeitachse

Lehrmodell · Zeit in Millisekunden · Verwaltungsaufwand nicht modelliert




**Notizen** 15 / 50

Die PlantUML-Abbildung ist ein Timing-Diagramm: Die horizontale Achse zeigt Zeitpunkte in Millisekunden, die Spur „Auftrag“ den Zustand einer einzelnen Ausführung. Bei $r = 2$ erfolgt die Freigabe. Der Auftrag wartet bis zum ersten Start $s = 4$. Von 4 bis 6 rechnet er, von 6 bis 8 wird er durch andere Arbeit unterbrochen, von 8 bis 9 rechnet er wieder. Bei $f = 9$ ist er fertig. Unterbrechung bedeutet hier, dass derselbe Kern zwischenzeitlich andere Arbeit ausführt; es gibt nicht zwei gleichzeitig laufende Arbeiten auf dieser Spur.
Die Fristspur markiert $d = 12$ als absolute Deadline. Daraus folgt die relative Deadline $D = d - r = 12 - 2 = 10$ Millisekunden. Die Pfeile unter der Auftragsspur bezeichnen einzelne Abschnitte, keine zusätzlichen Tätigkeiten. Der eigene Rechenbedarf zählt die beiden Laufabschnitte, die Antwortzeit dagegen die ganze Strecke von Freigabe bis Fertigstellung. Ein Verwaltungsaufwand für Eintritt und Rückkehr ist in diesem vereinfachten Modell nicht zusätzlich angesetzt. Reale Analysen müssen ihn passend zu ihrer Zählkonvention berücksichtigen. Nun berechnen wir die vier verschiedenen Dauern ausdrücklich.

**Folie** 16 / 50

### Rechenbedarf ist nicht Antwortzeit

Aus der Zeitachse: $r = 2$, $s = 4$, $f = 9$, $d = 12$ · jeweils in ms

- Eigener Rechenbedarf: $C = (6 - 4) + (9 - 8) = 3$ ms
- Startverzug: $s - r = 2$ ms
- Antwortzeit: $R = f - r = 7$ ms
- Fristreserve: $d - f = 3$ ms

Hier gilt $R \leq D$ mit $D = 10$ ms.

**Notizen** 16 / 50

$C$ bezeichnet hier den eigenen Bedarf an CPU-Zeit. CPU heißt Central Processing Unit, die zentrale Verarbeitungseinheit beziehungsweise der Prozessorkern. Gezählt werden nur die eigenen Laufabschnitte: $6 - 4 = 2$ und $9 - 8 = 1$ Millisekunden, insgesamt 3. Die Wartezeit von Freigabe $r = 2$ bis Start $s = 4$ beträgt 2 Millisekunden. Die Unterbrechung von 6 bis 8 fügt weitere 2 Millisekunden hinzu, ohne im Lehrmodell eigene Rechenarbeit zu sein.
Die Antwortzeit $R = f - r = 9 - 2 = 7$ Millisekunden umfasst alle drei Anteile: 2 Warten, 3 Rechnen und 2 Unterbrechung. Die absolute Deadline $d = 12$ ergibt mit $r = 2$ die relative Frist $D = 10$. Die Reserve $d - f = 12 - 9 = 3$ Millisekunden ist in diesem Lauf positiv. Ein negativer Wert würde eine Verspätung anzeigen. Die Beziehung $R \leq D$ prüft den Abschluss des ganzen Auftrags und kann daher nicht durch $C \leq D$ ersetzt werden. Der Rechenbedarf passt zwar in die Frist, aber zusätzliche Wartezeiten können trotzdem eine Verletzung verursachen. Diese Unterscheidung ist die Grundlage der nächsten Vorlesung.

**Folie** 17 / 50

### Periode und Frist beantworten andere Fragen

- Periode $T$: Abstand regelmäßiger Freigaben
- Relative Deadline $D$: zulässige Dauer ab einer Freigabe
- Lehrbeispiel: $T = 20$ ms, $D = 10$ ms
- Absolute Deadlines: $d_k = r_k + D$

Eine Aufgabe erzeugt mehrere einzelne Ausführungen: Jobs.

**Notizen** 17 / 50

Eine Task ist im theoretischen Aufgabenmodell eine wiederkehrende Aufgabe, etwa „Sortierauftrag aus dem Sensorstand berechnen“. Ein Job ist eine einzelne Ausführung dieser Aufgabe. Noch ist damit kein Betriebssystem-Thread gemeint. Für regelmäßige Freigaben bezeichnet $T$ die Periode, also den Zeitabstand zwischen zwei aufeinanderfolgenden Freigaben. $D$ ist die relative Deadline jedes einzelnen Jobs, unabhängig vom Abstand zum nächsten Job.
Im Lehrbeispiel liegen Freigaben bei $r_0 = 2$, $r_1 = 22$ und $r_2 = 42$ Millisekunden. Die Indizes 0, 1

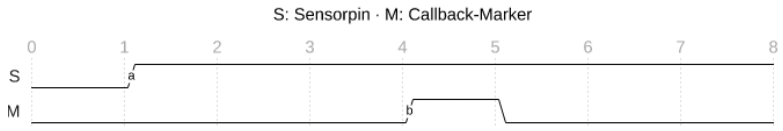
**Notizen, Fortsetzung** 17 / 50

und 2 nummerieren die Jobs. Mit $T = 20$ und $D = 10$ Millisekunden liegen ihre absoluten Deadlines $d_k = r_k + D$ bei 12, 32 und 52 Millisekunden. Ein Abschluss bei 18 ist für den ersten Job bereits zu spät, obwohl der zweite erst bei 22 freigegeben wird. Eine Periode legt deshalb nicht von selbst die zulässige Antwortzeit fest. Unregelmäßige Ereignisse haben keine feste Periode; gegebenenfalls wird ein kleinster Freigabeabstand spezifiziert. Eine solche Lastannahme wird später für die Zuteilung von Prozessorzeit benötigt.

**Folie** 18 / 50

## Latenz braucht zwei genaue Messpunkte

Lehrskizze · eine horizontale Stelle entspricht $1$ µs




Flanke bei 1 → Marker bei 4: $3$ µs

Pin → Marker umfasst mehr als den reinen Ausnahme-Eintritt.

**Notizen** 18 / 50

Latenz ist eine Verzögerung zwischen zwei benannten Ereignissen. Die WaveDrom-Abbildung zeigt digitale Pegel von links nach rechts. S bezeichnet den Sensorpin, M den Callback-Marker; die Legende über der Skala löst beide Kürzel auf. Eine horizontale Stelle entspricht im Lehrmodell einer Mikrosekunde, also einem Millionstel einer Sekunde. Der Sensorpin steigt bei Stelle 1 an, der Callback-Marker bei Stelle 4. Die Knotenbuchstaben a und b kennzeichnen beide steigenden Flanken; die Skala zählt die horizontalen Stellen. Die modellierte Pin-bis-Marker-Latenz beträgt $4 - 1 = 3$ Mikrosekunden. Der Marker fällt eine Stelle später wieder ab; seine Pulsbreite ist nicht die Latenz.

Ein Callback ist eine von der Hardwarebibliothek aufgerufene Anwendungsfunktion. Wird erst dort ein Ausgang gesetzt, enthält die gemessene Strecke Pin-Erkennung, Weiterleitung, Zulassung der Unterbrechung, Ausnahme-Eintritt und Software bis zum Marker. Interrupt-Latenz bis zur ersten Handlerinstruktion und Latenz bis zu diesem Callback-Marker sind deshalb unterschiedliche Größen. Auch der endgültige Stellauftrag kann nochmals später entstehen. Die Kurve ist ausdrücklich keine Boardmessung und macht keine Zusage über Eintrittszyklen eines konkreten Kerns. Eine einzelne Latenz beantwortet außerdem noch nicht, wie stark die Reaktion von Lauf zu Lauf schwankt.

**Folie** 19 / 50

### Konstanter Versatz ist noch kein Jitter

Lehrwerte derselben Latenzdefinition:

| Reihe | Latenzen in ms | Spanne |
|---|---|---|
| A | 5, 5, 5 | 0 ms |
| B | 4, 5, 6 | 2 ms |

Latenzjitter: $J_L = \max(L_i) - \min(L_i)$

**Notizen** 19 / 50

Jitter bezeichnet eine Schwankung; seine Metrik muss ausdrücklich genannt werden. Wir verwenden hier Peak-to-Peak, also die Spanne aus größtem minus kleinstem Wert. $L_i$ ist die Latenz des nummerierten Laufs $i$ zwischen denselben zwei Messpunkten. $J_L$ ist die beobachtete Latenzspanne der vorliegenden Reihe. Für A gilt $5 - 5 = 0$ Millisekunden, für B $6 - 4 = 2$ Millisekunden. Beide Reihen haben den Mittelwert 5 Millisekunden, aber unterschiedliche Schwankung.
Reihe A ist nicht verzögerungsfrei: Jede Antwort kommt um 5 Millisekunden später als das Bezugsereignis. Falls die Frist nur 3 Millisekunden beträgt, verletzt auch diese schwankungsfreie Reihe den Vertrag. Ein konstantes Zuspätkommen ist daher kein erfolgreicher Echtzeitnachweis. Umgekehrt sagt eine kleine Spanne nicht, ob der größte Wert innerhalb der Frist liegt. Alle Werte sind didaktische Annahmen. Die Spanne aus drei Läufen ist keine garantierte Schwankungsgrenze für alle Betriebsfälle. Bei wiederholten Ausgaben lässt sich zusätzlich die Schwankung ihrer Perioden untersuchen; dafür werden andere Differenzen gebildet.

**Folie** 20 / 50

### Periodenjitter und Phasenabweichung trennen

Sollperiode $T = 10$ ms · alle Zeitpunkte in ms

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| Soll | 0 | 10 | 20 | 30 | 40 |
| Lehrwerte Ist | 0 | 9,8 | 20 | 29,8 | 40 |

- Istperioden: 9,8 / 10,2 / 9,8 / 10,2 ms
- Periodenspanne: $0{,}4$ ms · Phasenspanne: $0{,}2$ ms

**Notizen** 20 / 50

Die Spaltennummern 0 bis 4 bezeichnen aufeinanderfolgende Ausgaben. Die Sollzeitpunkte folgen einer

**Notizen, Fortsetzung** 20 / 50

Periode $T = 10$ Millisekunden. Die angenommenen Istzeitpunkte sind keine Hardwaremesswerte. Eine Istperiode entsteht durch Subtraktion benachbarter Istzeitpunkte: $9{,}8 - 0 = 9{,}8$, $20 - 9{,}8 = 10{,}2$, $29{,}8 - 20 = 9{,}8$ und $40 - 29{,}8 = 10{,}2$ Millisekunden. Die Peak-to-Peak-Spanne dieser Perioden beträgt $10{,}2 - 9{,}8 = 0{,}4$ Millisekunden.
Die Phasenabweichung vergleicht dagegen den jeweiligen Istzeitpunkt mit seinem Sollzeitpunkt. Es ergeben sich 0, −0,2, 0, −0,2 und 0 Millisekunden. Negative Werte bedeuten eine frühere Ausgabe. Die Spanne dieser Abweichungen ist $0 - (-0{,}2) = 0{,}2$ Millisekunden. Beide Ergebnisse sind korrekt, beschreiben aber verschiedene Größen. Eine Antwort „Jitter ist 0,2“ ohne Metrik und Einheit ist deshalb unvollständig. Auch eine systematisch andere Periode muss von einer Schwankung getrennt werden. Eine einzelne Spur genügt zur Periodenberechnung, für Ereignis-bis-Reaktion-Latenzen werden zusätzlich zugeordnete Eingangspunkte benötigt.

**Folie** 21 / 50

## Messmaximum ist keine sichere Schranke

| Größe | Bedeutung |
|---|---|
| Messmaximum | Größter bisher beobachteter Wert |
| WCET | Worst-Case Execution Time unter benannten Annahmen |
| Angesetzte WCET-Schranke | Begründete obere Grenze des eigenen Rechenbedarfs |

Lehrannahme: Maximum $80$ µs; gegebene Schranke $120$ µs.

**Notizen** 21 / 50

WCET heißt Worst-Case Execution Time, die größte eigene Ausführungszeit innerhalb eines benannten Betriebsmodells. Für die Analyse setzt man eine sichere obere Schranke an; sie kann größer als das tatsächliche, unbekannte Maximum sein. Das Messmaximum ist lediglich der größte Wert der bisher untersuchten Läufe. Ungetestete Eingaben, andere Speicherzustände oder zusätzliche Buslast können längere Ausführungen verursachen.
Die 80 und 120 Mikrosekunden sind Lehrannahmen. Die 120 werden hier als bereits begründete Schranke gegeben, nicht aus $80 + 40$ abgeleitet. Eine frei gewählte Reserve beweist keine sichere Grenze. Für eine reale Schranke braucht man eine passende Analyse oder belastbare Messmethodik mit ihrer begründeten Abdeckung und ihren Annahmen. Zudem muss die Zählkonvention klar sein: Wir zählen eigenen Rechenbedarf und keine Wartezeit auf andere Arbeiten. Eine Zeitmessung um eine Funktion kann Unterbrechungen enthalten und ist dann nicht automatisch eine Messung dieses Bedarfs. Eine Grenze für den Rechenbedarf allein ist schließlich noch keine Grenze für die ganze Antwortzeit. Die Hardware- und Lastannahmen machen diese Abgrenzungen konkret.

**Folie** 22 / 50

## Die Konfiguration gehört zum Zeitargument

- Eingaben und durchlaufene Programmpfade
- Übersetzungsoptionen und tatsächliche Takte
- Speicherzugriffe, Cache und Buslast
- Unterbrechungen und Sperrzeiten
- Messpunkte, Messaufwand und Zahl der Läufe

**Notizen** 22 / 50

Die Laufzeit einer Funktion hängt nicht nur von ihrer Zeilenzahl ab. Unterschiedliche Eingaben können andere Zweige und Schleifenzahlen wählen. Der Compiler, also das Übersetzungsprogramm, erzeugt je nach Optimierungsoptionen unterschiedliche Maschinenbefehle. Der tatsächliche Kerntakt und die Speicherwartezeiten bestimmen deren Dauer. Ein Cache ist ein schneller Zwischenspeicher; ein gefundener Wert kann schneller erreichbar sein als ein neu zu ladender. Mehrere Beteiligte auf einem Bus können sich bei Zugriffen verzögern.
Ein Interrupt unterbricht den normalen Ablauf. Seine Laufzeit verlängert eine verstrichene Funktionsdauer; eine Maskierung, also das zeitweise Sperren bestimmter Unterbrechungen, verzögert deren Behandlung. Auch Messmarkierungen verbrauchen Zeit. Deshalb gehören Anfangs- und Endpunkt, Anzahl der Läufe und Messauflösung in das Protokoll. Wir müssen nicht schon alle Hardwaredetails berechnen, aber wissen, welche Angaben eine pauschale Aussage „diese Funktion dauert immer so lange“ benötigt. Ein Kern ohne Datencache liefert ebenfalls nicht automatisch identische oder ausreichend kurze Laufzeiten. Vorhersagbarkeit ist eine begründete Aussage über einen festgelegten Betriebsbereich.

**Folie** 23 / 50

## Determinismus heißt hier: begrenzt vorhersagbar

- Laufzeiten dürfen zwischen Ausführungen variieren
- Ihre relevanten Grenzen müssen begründet sein
- Niedriger Mittelwert und ruhiges Blinken reichen nicht
- Fehlende Beobachtung einer Verletzung ist kein Beweis

Eine Fristprüfung braucht die ganze Antwortkette.

**Notizen** 23 / 50

Zeitliche Vorhersagbarkeit bedeutet im Kurs, dass wir die relevanten Zeitgrößen unter dokumentierten Bedingungen begrenzen können. Die Laufzeit muss dafür nicht bei jedem Durchlauf identisch sein. Eine Funktion mit unterschiedlichen Pfaden kann eine sichere obere Ausführungszeitschranke besitzen. Ein streng wiederholter, aber zu langsamer Ablauf kann dagegen jede Deadline, also jede gesetzte Frist, verletzen.
Der Begriff Determinismus wird in anderen Zusammenhängen auch für eindeutige Ergebnisse bei gleicher

**Notizen, Fortsetzung** 23 / 50

Eingabe verwendet. Hier benennen wir ausdrücklich die zeitliche Bedeutung. Eine nachgewiesene obere Grenze muss alle im Vertrag enthaltenen Verzögerungen berücksichtigen. Zu einer kurzen eigenen Berechnung kommen möglicherweise Eingangsverzögerung, Warten, Unterbrechungen und Ausgabe. Optisch regelmäßiges Blinken ist eine grobe Beobachtung mit sehr begrenzter zeitlicher Auflösung. Es zeigt weder alle Periodenabweichungen noch einen maximalen Rechenbedarf. Die folgende Verständnisfrage prüft deshalb, ob die eigene Rechenzeit und die vollständige Antwortzeit bereits auseinandergehalten werden.

**Folie** 24 / 50

### Zwischencheck: reicht kurze Rechenzeit?

Lehrfall mit $D = 10$ ms:

- Freigabe bei $r = 0$ ms
- Warten bis $s = 9$ ms
- Eigene Rechnung ohne Unterbrechung: $C = 2$ ms

Berechnen Sie $f$, $R$ und die Fristreserve.

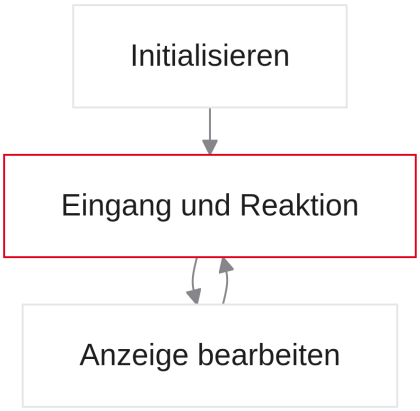
**Notizen** 24 / 50

Die Musterlösung beginnt mit den Endpunkten. Die Freigabe liegt bei $r = 0$ Millisekunden, der erste Start bei $s = 9$. Der eigene Rechenbedarf $C = 2$ Millisekunden wird danach ohne Unterbrechung abgearbeitet. Also liegt die Fertigstellung bei $f = s + C = 9 + 2 = 11$ Millisekunden. Die Antwortzeit $R = f - r = 11 - 0 = 11$ Millisekunden ist größer als die relative Deadline $D = 10$.
Die absolute Deadline ist $d = r + D = 10$ Millisekunden. Die Fristreserve beträgt $d - f = 10 - 11 = -1$ Millisekunde: Der Auftrag ist eine Millisekunde verspätet. Obwohl $C < D$ gilt, ist die Frist verletzt. Die CPU, Central Processing Unit beziehungsweise der Prozessorkern, kann die zwei Millisekunden Rechnung schnell genug erledigen, hat sie aber zu spät begonnen. Eine Musterantwort muss die Wartezeit mitzählen. Dieser Fall stellt die Verbindung zur Programmstruktur her: Eine lange andere Funktion oder eine gesperrte Unterbrechung kann einen solchen Startverzug erzeugen. Wir untersuchen nun, welche Wartezeiten einfache Kontrollflussmuster entstehen lassen.

**Folie** 25 / 50

## Bare Metal: eine Hauptschleife

Initialisieren, dann Eingang und Anzeige wiederholen




**Notizen** 25 / 50

Bare Metal bezeichnet hier eine Anwendung ohne Betriebssystemkernel, also ohne zentrale Betriebssystemverwaltung ihrer Arbeiten. Bibliotheken, Zeitgeber und Unterbrechungen sind trotzdem möglich. Eine Super-Loop ist eine Endlosschleife, die mehrere Funktionen nacheinander aufruft. Im Diagramm wird zuerst einmal initialisiert. Dann folgt wiederholt die Eingangsprüfung mit gegebenenfalls einer Reaktion und die Anzeigenbearbeitung.
Die Leserichtung ist von oben nach unten; der Rückpfeil bedeutet die nächste Runde. Die rot umrandete Eingangsprüfung ist für unseren Zeitvertrag wichtig. Sie kommt erst wieder an die Reihe, wenn die Anzeigenfunktion zurückkehrt. Die Kästen sind Arbeitsschritte auf einem Kern, keine parallel rechnenden Prozesse. Die Größen der Kästen enthalten keine Zeitinformation. Wir nehmen zunächst an, dass weder Unterbrechungen noch ein zweiter Anwendungskern die Reihenfolge verändern. Das Muster hat wenig Verwaltungsaufwand und einen leicht verständlichen Ablauf. Ob es die Frist erfüllt, hängt aber von den maximalen Dauern der Funktionen und dem Verhalten des Eingangssignals ab. Der folgende Lehrcode macht die Abfragestelle sichtbar.

**Folie** 26 / 50

### Polling im C-Kontrollfluss

Lehrcode mit angenommenen Hilfsfunktionen · kein Boardprogramm

```
for (;;) {
    if (SensorauftragLiegtVor()) {
        StellauftragAusgeben();
    }
    AnzeigeBearbeiten();
}
```

Die nächste Prüfung wartet auf die Rückkehr der Anzeige.

**Notizen** 26 / 50

Polling bedeutet, dass die Anwendung aktiv nachfragt, ob ein Ereignis vorliegt. Die C-Schleife `for (;;)` hat keine Abbruchbedingung und wiederholt ihren Rumpf dauerhaft. `SensorauftragLiegtVor` ist eine hier angenommene Prüffunktion mit einem Wahrheitswert als Ergebnis. `if` führt die eingerückte Reaktion nur bei einem vorliegenden Auftrag aus. `StellauftragAusgeben` erledigt im Modell die geforderte Ausgabe. `AnzeigeBearbeiten` kehrt erst nach Abschluss ihrer aktuellen Arbeit zurück.
Diese drei Hilfsfunktionen sind ausdrücklich Lehrcode und kein vollständiges Firmwareprojekt. Ihre Dauer und die Speicherung eines Eingangsevents müssen für eine Zeitrechnung separat festgelegt werden. Wenn während der Anzeigenbearbeitung ein Auftrag eintrifft, prüft die Schleife ihn erst in der nächsten Runde. Die CPU, Central Processing Unit oder Prozessorkern, rechnet dabei durchaus; sie rechnet nur an anderer Arbeit. Die Schleife kann bei kurzen begrenzten Abschnitten gute Zeitgrenzen haben. Eine Endlosschleife ist daher nicht grundsätzlich ein Echtzeitfehler. Problematisch wird die unbegrenzt oder zu lange ausbleibende Rückkehr zur wichtigen Abfrage.

**Folie** 27 / 50

### Eine begrenzte Polling-Runde kann reichen

Lehrannahmen: Ereignis bleibt gespeichert, keine Unterbrechungen

| Arbeit je Runde | Maximale Dauer |
|---|---|
| Eingangsprüfung | 0,1 ms |
| Reaktion bei Auftrag | 0,2 ms |
| Anzeige | 5,0 ms |

Runde höchstens $5{,}3$ ms · sichere Modellabschätzung $R \leq 5{,}5$ ms

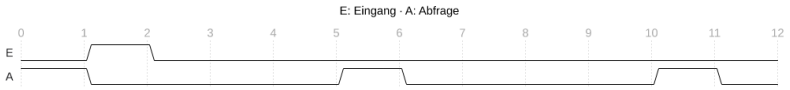
**Notizen** 27 / 50

Wir geben für jeden Arbeitsschritt eine maximale Dauer vor: Eingangsprüfung 0,1, Reaktion 0,2 und Anzeigenarbeit 5,0 Millisekunden. Die maximale Runde beträgt damit $0{,}1+0{,}2+5{,}0 = 5{,}3$ Millisekunden. Das Eingangsevent bleibt bis zur nächsten Abfrage gespeichert; die Erkennung liegt im Modell jeweils am Ende der Eingangsprüfung. Es gibt keine Unterbrechungen und höchstens einen relevanten Auftrag. Ohne diese Annahmen beantwortet die Summe die Fristfrage nicht vollständig.
Trifft das Ereignis gerade nach der zugehörigen Prüfung ein, muss es bis zur nächsten Erkennung warten. Wir begrenzen dieses Warten konservativ durch die ganze maximale Runde von 5,3 Millisekunden. Danach kostet die eigene Reaktion weitere 0,2 Millisekunden. Somit gilt die sichere, gegebenenfalls nicht engste Modellabschätzung $R \leq 5{,}3 + 0{,}2 = 5{,}5$ Millisekunden. $R$ ist die vollständige Antwortzeit; die relative Deadline $D = 10$ Millisekunden wäre in diesem Modell erfüllbar. Benötigt die Anzeige dagegen 50 Millisekunden, wird die Runde 50,3 und die gleiche Abschätzung 50,5 Millisekunden. Die günstige Rechnung ist dann kein Nachweis mehr; ein Ereignis unmittelbar vor langer Anzeigenarbeit kann die Frist tatsächlich verletzen. Polling ist folglich eine Frage passender Grenzen.

**Folie** 28 / 50

## Kurzer Puls: verspätet oder ganz verloren?

Lehrmodell · eine Stelle entspricht $1$ ms




Ein Pegelpoll sieht den Puls zwischen den Abfragen nicht.

**Notizen** 28 / 50

Die WaveDrom-Kurve zeigt zwei digitale Spuren auf derselben Zeitachse: E ist der Eingang, A die Abfrage. Die Legende über der Skala erklärt die beiden Kürzel. Eine Stelle entspricht im Lehrmodell einer Millisekunde. Der Eingang steigt bei 1 an und fällt bei 2 wieder ab. Die kurze Markierung „Abfrage“ steht bei 0, 5 und 10 Millisekunden für das jeweilige Lesen des Pegels. An diesen Zeitpunkten ist der Eingang niedrig. Reines Pegelpolling erkennt den dazwischenliegenden Puls deshalb überhaupt nicht.
Das ist nicht bloß eine große Antwortzeit, sondern ein Informationsverlust: Es gibt keinen später erkannten Auftrag. Ein Hardware-Latch, also ein festhaltendes Ereignismerkmal, könnte den Puls bis zur Abfrage speichern. Ein Puffer könnte zusätzlich mehrere Ereignisse aufbewahren, hätte aber eine endliche Kapazität. Die Annahme der vorigen Rechnung „Ereignis bleibt gespeichert“ ist somit eine echte technische Voraussetzung. Auch eine Interruptanforderung kann je nach Peripherie mehrere Ereignisse zu einem einzigen anstehenden Merkmal zusammenfassen. Eine schnellere Programmstruktur allein garantiert keine vollständige Ereigniszählung. Für längere gespeicherte Ereignisse bleibt wiederum die Wartezeit der Hauptschleife zu prüfen.

**Folie** 29 / 50

## Lange Arbeit in kurze Schritte zerlegen

Lehrmodell: Anzeige in fünf Schritte zu je höchstens $1$ ms zerlegen

- Nach jedem Schritt Eingang prüfen
- Fortschritt der Anzeige zwischen Runden speichern
- Maximale Runde: $0{,}1 + 0{,}2 + 1{,}0 = 1{,}3$ ms
- Konservative Antwortgrenze: $R \leq 1{,}5$ ms

Die Gesamtanzeige dauert weiterhin mehrere Runden.

**Notizen** 29 / 50

Eine lange Funktion muss nicht immer am Stück laufen. Im Lehrmodell speichern wir, welcher Teil der Anzeige schon bearbeitet wurde, und erledigen pro Runde nur den nächsten Schritt von höchstens einer Millisekunde. Anschließend kann die Hauptschleife wieder zum Eingang zurückkehren. Dieses freiwillige Zurückgeben des Kontrollflusses heißt kooperativ. Es setzt voraus, dass der Schritt wirklich eine begrenzte Dauer hat und der gespeicherte Fortschritt korrekt ist.
Mit Eingangsprüfung 0,1, gegebenenfalls Reaktion 0,2 und einem Anzeigeschritt 1,0 Millisekunden ergibt sich eine maximale Runde von 1,3 Millisekunden. $R$ bezeichnet die vollständige Antwortzeit von Freigabe bis zur geforderten Ausgabe. Wie zuvor begrenzen wir das Warten konservativ durch eine ganze Runde und addieren die Reaktionsdauer: $R \leq 1{,}3 + 0{,}2 = 1{,}5$ Millisekunden. Die Gesamtanzeige ist damit nicht plötzlich in einer Millisekunde fertig; sie benötigt fünf Schritte und mehrere Runden. Mögliche Verwaltungskosten müssen in den gegebenen Schrittgrenzen enthalten sein. Die Zerlegung verbessert hier die Reaktionsmöglichkeit ohne Betriebssystem. Ein Hardwareereignis kann den Ablauf dagegen auch ohne freiwillige Rückkehr unterbrechen. Diesen zweiten Mechanismus führen wir nun ein.

**Folie** 30 / 50

## Interrupt: Hardware meldet ein Ereignis

- IRQ: Interrupt Request, eine Unterbrechungsanforderung
- ISR: Interrupt Service Routine, die Behandlungsroutine
- Zulässiger IRQ kann die Hauptschleife unterbrechen
- Kern sichert Zustand und setzt ihn nachher fort
- Eine Anforderung bedeutet noch keinen sofortigen Abschluss

**Notizen** 30 / 50

Ein Interrupt ist eine Unterbrechung des normalen Kontrollflusses zur Behandlung eines Ereignisses. IRQ heißt Interrupt Request und bezeichnet die Anforderung, ISR heißt Interrupt Service Routine und bezeichnet die ausgeführte Behandlungsroutine. Eine Peripherieeinheit, etwa ein Timer, meldet ein Ereignis. Der Kern entscheidet anhand seiner Zulassungsregeln, ob und wann er die zugehörige Routine startet.

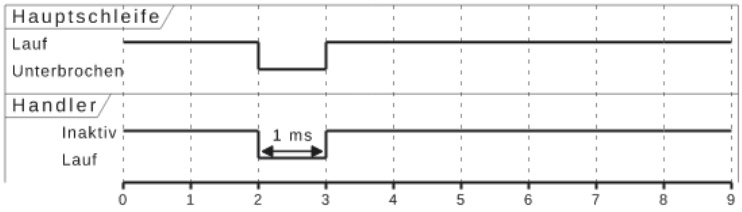
**Notizen, Fortsetzung** 30 / 50

Beim Ausnahme-Eintritt sichert die Hardware den erforderlichen Zustand. Nach dem Handler wird die unterbrochene Arbeit fortgesetzt. Auf einem einzelnen Kern rechnen Hauptschleife und Handler deshalb nicht gleichzeitig, sondern zeitlich verschränkt. Der Handler ist auch kein neuer Betriebssystem-Task. Zu seinem Start können Pin-Erkennung, Weiterleitung, Maskierung oder dringendere Handler Zeit hinzufügen. Ein Interrupt kann somit das Warten auf das Ende einer Anzeigenfunktion vermeiden, falls er währenddessen zulässig ist. Er liefert aber keine allgemeine Fristgarantie und speichert nicht automatisch beliebig viele Ereignisse. Eine kurze definierte Arbeit im Handler ist der einfachste Fall; eine längere gesamte Antwort braucht ein zusätzliches Ablaufkonzept.

**Folie** 31 / 50

## Hintergrund wird unterbrochen und fortgesetzt

Ein-Kern-Lehrmodell · Zeit in ms




Basisframe: `R0`–`R3`, `R12`, `LR`, `PC`, `xPSR`

**Notizen** 31 / 50

Die beiden Spuren der PlantUML-Zeitachse zeigen Ausführungszustände auf demselben Kern. Von 0 bis 2 Millisekunden läuft die Hauptschleife. Bei 2 wird ein zulässiger Interrupt angenommen; im Lehrmodell liegen Anforderung und Start an derselben Stelle. Die Hauptschleife ist bis 3 unterbrochen, während der Handler eine Millisekunde rechnet. Danach ist der Handler inaktiv und die Hauptschleife läuft weiter. Zu keinem Zeitpunkt haben beide Spuren den Zustand „Lauf“ gleichzeitig.
Die Abbildung idealisiert Eintritt und Rückkehr; deren Kosten sind hier nicht zusätzlich eingezeichnet. In der Realität gehören sie in die passend definierte Zeitbilanz. Eine Funktion in der Hauptschleife benötigt wegen der Unterbrechung mehr verstrichene Zeit, obwohl ihr eigener Rechenbedarf im einfachen Modell unverändert bleibt. Deshalb darf die Pulsbreite um eine Hintergrundfunktion nicht ohne Weiteres als reine CPU-Zeit gelesen werden. CPU bedeutet Central Processing Unit, den Prozessorkern. Der Vorteil des Handlers liegt in der früheren Ausführung seiner kleinen Reaktion. Ob diese Reaktion die vollständige geforderte Antwort ist, muss der Zeitvertrag entscheiden.
Beim Cortex-M4 legt der Ausnahme-Eintritt ohne zusätzlichen Gleitkommakontext einen Basisframe im Stack ab. Stack bedeutet Stapelspeicher für Rückkehr- und lokale Informationen; Frame ist ein zusammengehöriger gesicherter Block. `R0` bis `R3` sind Arbeitsregister, unter anderem für Argumente und Zwischenwerte, `R12` ein weiteres temporäres Arbeitsregister. `LR` heißt Link Register und enthält Rückkehrinformation, `PC` Program Counter und enthält eine Befehlsadresse. `xPSR` heißt extended Program Status Register und hält Prozessorstatus, etwa Bedingungsmerker und Ausnahmeinformation. Register sind kleine Speicher direkt im Kern. Diese acht 32-Bit-Worte ergeben im Basisframe $8 \cdot 4 = 32$ Byte; zusätzliche Ausrichtung oder Gleitkommasicherung kann weiteren Platz benötigen. Damit kann der Kern den unterbrochenen Kontext fortsetzen. Dieser Hardwareframe allein ist kein vollständiger Wechsel

**Notizen, Fortsetzung** 31 / 50

zu einer anderen Betriebssystem-Task. Eintritts- und Rückkehrkosten sind in der gezeichneten Zeitachse vereinfacht; eine feste Pin-bis-Callback-Zeit folgt nicht daraus.

**Folie** 32 / 50

## Wer darf einen Handler unterbrechen?

- NVIC: Nested Vectored Interrupt Controller
- Verwaltet anstehende Anforderungen und Prioritäten
- Kleinere Prioritätszahl bedeutet höhere Dringlichkeit
- Maskierung oder höherer Handler können verzögern
- IRQ-Priorität ist noch keine Betriebssystem-Taskpriorität

**Notizen** 32 / 50

NVIC steht für Nested Vectored Interrupt Controller, die verschachtelungsfähige Interruptsteuerung des Cortex-M-Kerns. „Vectored“ bedeutet, dass die Anforderung über eine Tabelle ihrer Behandlungsroutine zugeordnet wird. „Nested“ bedeutet, dass ein ausreichend dringender zulässiger Handler einen bereits laufenden Handler unterbrechen kann. IRQ heißt Interrupt Request, die Anforderung. Im hier verwendeten Prioritätssystem ist eine kleinere Zahl dringender als eine größere.
Eine anstehende, englisch pending, Anforderung ist noch nicht unbedingt aktiv. Bestimmte Anforderungen können maskiert, also gesperrt sein. Ein bereits laufender höher priorisierter Handler kann die Annahme ebenfalls verzögern. Die genaue Zahl implementierter Prioritätsbits und ihre Gruppierung gehören zur konkreten Kern- und Projektkonfiguration. Wir benötigen heute keine pauschale Kanal- oder Zykluszahl. Die NVIC-Priorität steuert die Hardwarehandler, während später ein Betriebssystem andere Prioritäten für seine Tasks verwenden kann. Beide Zahlenräume dürfen nicht einfach gleichgesetzt werden. Für das Verständnis der Verschachtelung unterscheiden wir nun Unterbrechungsrang und Reihenfolge gleichzeitig wartender Anforderungen.

**Folie** 33 / 50

## Preemption Priority und Subpriority

| Feld | Aufgabe |
|---|---|
| Preemption Priority | Darf eine laufende ISR unterbrechen? |
| Subpriority | Reihenfolge gleichzeitig anstehender zulässiger IRQs |

Gleicher Unterbrechungsrang: kleinere Subpriority präemptiert nicht.

**Notizen** 33 / 50

Preemption Priority ist die Präemptionspriorität, also der Rang für eine erzwungene Unterbrechung. Subpriority ist die Unterpriorität für die Auswahl zwischen gleichzeitig anstehenden zulässigen Anforderungen gleichen Präemptionsrangs. ISR bedeutet Interrupt Service Routine, die Behandlungsroutine; IRQ bedeutet Interrupt Request, die Anforderung. Welche Bits auf beide Felder entfallen, hängt von der gewählten Prioritätsgruppierung ab. Ohne zugewiesene Subpriority-Bits spielt dieses Feld keine Rolle.
Angenommen, zwei Handler besitzen Präemptionspriorität 2, aber Subpriority 0 und 1. Solange einer davon läuft, unterbricht ihn der andere nicht allein wegen seiner kleineren Subpriority. Stehen beide zur Auswahl an, wird im Modell mit entsprechender Gruppierung der mit Subpriority 0 zuerst gewählt. Ein zulässiger Handler mit Präemptionspriorität 1 kann dagegen einen mit Rang 2 unterbrechen. Die Zahlen sind Lehrwerte, keine konkrete Timerkonfiguration des Labors. Diese Unterscheidung verhindert, dass eine geänderte Unterpriorität fälschlich als Mittel gegen eine lange Sperrzeit benutzt wird. Der wirksamste erste Entwurfsschritt bleibt, Handlerarbeit klein und begrenzt zu halten.

**Folie** 34 / 50

## Kurze Handler lassen Zeit für andere Arbeit

- Ereignis behandeln und erforderliche Quelle quittieren
- Nur kleine, begrenzte Reaktion im Handler
- Lange Auswertung und Ausgabe in den Hintergrund
- Keine wartenden Delays oder blockierende Textausgabe
- Dauer und zulässige Ereignisrate gehören ins Budget

**Notizen** 34 / 50

Ein Handler ist die Behandlungsfunktion einer Unterbrechung. Quittieren bedeutet, das zugehörige Ereignismerkmal der Peripherie nach deren Regeln zurückzusetzen, damit die Quelle korrekt weiterarbeiten kann. Die verwendete Hardwarebibliothek übernimmt einen Teil dieser Schritte; ein zusätzlicher falscher Registerzugriff kann Ereignisse verlieren. In unserer Anwendung prüft der Timer-Callback, die von der Hardwarebibliothek aufgerufene Anwendungsfunktion, deshalb nur seine Instanz und schaltet einen Ausgang um.
Lange Handler verzögern die Hauptschleife und weniger dringliche Handler. Ein wartender Delay-Aufruf kann im Handler sogar hängen bleiben, wenn der benötigte Zeitgeber dort nicht fortschreitet. Blockierende Textausgabe kann auf eine Schnittstelle warten. Solche Arbeit gehört nicht in den heutigen Callback. Nichtblockierende Speziallösungen haben eigene Verträge und werden hier nicht vorausgesetzt. Auch viele kurze Handler können insgesamt überlasten: Die Dauer pro Ereignis und die maximal erlaubte Ereignisrate müssen zusammenpassen. Wenn eine längere Auswertung erforderlich ist, kann der Handler nur den Auftrag erfassen. Dann müssen dessen Daten korrekt an den Hintergrund übergeben werden; eine gemeinsame Variable löst diese Übergabe nicht automatisch sicher.

**Folie** 35 / 50

### Ein gemeinsames Flag kann ein Ereignis verlieren

Lehrfolge auf einem Kern:

| Schritt | Flag |
|---|---|
| Erstes Ereignis setzt das Flag | 1 |
| Main liest: Auftrag liegt vor | 1 |
| Zweites Ereignis setzt es erneut | 1 |
| Main löscht das alte Merkmal | 0 |

`volatile` macht Lesen und Löschen nicht zu einem unteilbaren Austausch.

**Notizen** 35 / 50

Ein Flag ist eine Merker-Variable, hier mit den Werten 0 und 1. Main bezeichnet den Ablauf der Hauptschleife. Eine Race Condition, deutsch ein Wettlauf, entsteht, wenn das Ergebnis von der zeitlichen Verschränkung mehrerer Zugriffe abhängt. Die Tabelle zeigt eine mögliche Folge: Das erste Ereignis setzt 1, Main erkennt diesen Auftrag, dann kommt ein zweites Ereignis. Sein erneutes Schreiben von 1 unterscheidet sich im gespeicherten Wert nicht vom ersten. Wenn Main anschließend 0 schreibt, verschwindet auch die Information über das zweite Ereignis.
`volatile` ist ein C-Typzusatz, der die erforderlichen Zugriffe auf ein außerhalb des normalen Ablaufs veränderliches Objekt erhält. Er schützt diese mehrschrittige Operation nicht vor Unterbrechung und zählt keine Ereignisse. Ein kurzer geschützter Austausch kann die Lücke zwischen Lesen und Löschen schließen, sperrt dabei aber bestimmte Unterbrechungen. Auch danach kann ein einzelnes Flag mehrere eintreffende Ereignisse zusammenfassen. Für jedes einzeln benötigte Ereignis ist daher eine geeignete Zähl- oder Pufferstrategie nötig. Deren vollständige Umsetzung folgt später; der heutige LED-Callback übergibt kein gemeinsames Flag an Main.

**Folie** 36 / 50

## Vordergrund und Hintergrund

Früh erfassen, später bearbeiten




**Notizen** 36 / 50

Foreground bedeutet hier Vordergrund und bezeichnet die kurze Interruptarbeit. Background bedeutet Hintergrund und bezeichnet die Hauptschleife. Das Diagramm beschreibt einen Daten- und Arbeitsfluss, keine garantierten Zeitabstände: Hardwareereignis, kurze Erfassung im Handler, gespeicherter Auftrag, spätere Hauptbearbeitung und geforderter Abschluss. Der rot umrandete Auftrag markiert die Übergabe, deren Datenkonsistenz und Kapazität geklärt werden müssen.
Die frühe Erfassung vermeidet das Warten auf die nächste Eingangsprüfung. Muss die vollständige Sortierentscheidung jedoch in Main, der Hauptschleife, erfolgen, kann deren lange Anzeigenarbeit die gesamte Antwort weiterhin verzögern. Eine kurze Latenz bis zum Handler ist also keine kurze Antwortzeit bis zum Abschluss. Falls die ganze kleine Reaktion im Handler erledigt werden kann, wie das Umschalten einer Leuchtdiode, entfällt diese nachgelagerte Wartezeit. Das Muster eignet sich für überschaubare Anwendungen, aber viele Aufträge und Prioritäten erfordern mehr Verwaltung. Damit ist das Problem vorbereitet, das ein späteres Echtzeitbetriebssystem strukturiert: die kontrollierte Auswahl und Fortsetzung mehrerer Arbeiten.

**Folie** 37 / 50

## Warum später ein Echtzeitbetriebssystem?

- RTOS: Real-Time Operating System
- Verwaltet getrennte Tasks und deren Ausführungszustand
- Scheduler wählt ausführungsbereite Arbeit
- Bietet Hilfen für Zusammenarbeit und Kommunikation
- Die Zeitgarantie benötigt weiterhin einen Nachweis

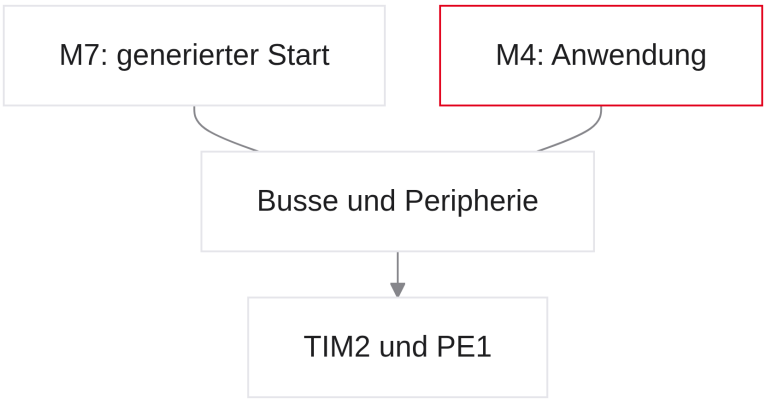
**Notizen** 37 / 50

RTOS steht für Real-Time Operating System, Echtzeitbetriebssystem. Ein Kernel ist dessen zentraler Verwaltungsbaustein. Ein Scheduler teilt Prozessorzeit zu, indem er aus ausführungsbereiten Arbeiten auswählt. Eine Betriebssystem-Task ist ein verwalteter Ausführungspfad mit eigenem gespeicherten Kontext, also dem Zustand, mit dem sie später fortgesetzt werden kann. Das ist mehr als ein Funktionsaufruf, der erst nach vollständiger Bearbeitung zurückkehrt.
Ein solches System kann die Hauptschleife mit vielen Merker-Abfragen übersichtlicher strukturieren und Kommunikations- oder Schutzfunktionen anbieten. Es macht unbekannte Laufzeiten, Überlast und falsche Datenübergaben aber nicht automatisch korrekt. FreeRTOS ist das im weiteren Kurs verwendete Echtzeitbetriebssystem. Heute wird es nicht eingeschaltet; der kleine Bare-Metal-Versuch, also die Anwendung ohne Kernel, soll zunächst den Unterschied zwischen eigenem Rechnen und Warten sichtbar machen. Die nächste Vorlesung untersucht die Zuteilungsregeln im theoretischen Task-Modell. Vor dem heutigen Versuch müssen wir außerdem festhalten, welcher der zwei Kerne das Programm und welche Ressourcen es ausführt.

**Folie** 38 / 50

## Board und Anwendungskern

NUCLEO-H755ZI-Q · Mikrocontroller STM32H755ZIT6




**Notizen** 38 / 50

NUCLEO-H755ZI-Q ist der Boardname; STM32H755ZIT6 ist die Bezeichnung des darauf verwendeten Mikrocontrollers. Cortex-M7 und Cortex-M4 sind seine zwei Prozessorkerne. Für den heutigen Versuch gehört die Anwendung zum Cortex-M4, kurz M4. Der M7 behält den vorbereiteten generierten Start- und Systemanteil. TIM2 bezeichnet Timerinstanz 2, PE1 den Ausgang Port E, Pin 1. Dort liegt beim beschriebenen MB1363-Board die gelbe Leuchtdiode LD2. Die Bestückung des konkreten Boards wird am Boardhandbuch geprüft.
Die Abbildung ist ein Architekturdiagramm. Die Linien zeigen die gemeinsame Infrastruktur und die Zuordnung der Ressourcen, nicht parallele Handler oder unabhängige Timerkopien. Der markierte M4 ist der einzige Anwendungseigentümer von TIM2 und PE1. Der M7 darf sie nicht zusätzlich konfigurieren oder umschalten. Beide Kerne teilen Busse und können sich bei Zugriffen beeinflussen; zwei Kerne bedeuten deshalb nicht automatisch zwei störungsfreie Rechner. Die Lehrtaktraten sind für den Cortex-M7 400 Megahertz und für den Cortex-M4 200 Megahertz. Dies sind konfigurierte Lehrannahmen, keine

**Notizen, Fortsetzung** 38 / 50

Messwerte oder automatisch erzeugten Einstellungen. Die tatsächlichen Frequenzen werden am Projekt dokumentiert, nicht aus zulässigen Maximalwerten abgeleitet. Die Bootvorbereitung muss beide Images in den vorgesehenen Zustand bringen.

**Folie** 39 / 50

## Den generierten Dual-Core-Start erhalten

Häufiges Cube-Template · konkrete Projektvariante prüfen

| Kern | Startaufgabe |
|---|---|
| M4 | Boot-Benachrichtigung aktivieren, in D2-Stop warten |
| M7 | Gemeinsame Takte initialisieren, Boot-Freigabe geben |
| M4 danach | Eigene Peripherie und Anwendung starten |

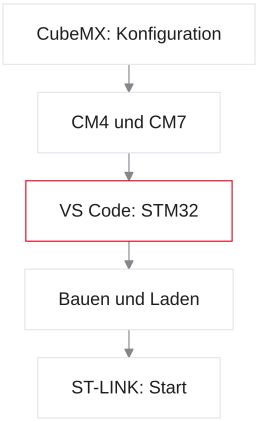
Boot-Freigabe und spätere Datenübergabe sind verschiedene Aufgaben.

**Notizen** 39 / 50

Dual Core bedeutet zwei Kerne. Cube bezeichnet die Werkzeugfamilie von STMicroelectronics für Konfiguration und generierten Startcode. In einem häufig verwendeten Dual-Core-Template führt der Cortex-M4, kurz M4, zunächst Bootcode aus, aktiviert eine Benachrichtigung und wartet im D2-Stop-Zustand. D2 bezeichnet eine Takt- und Leistungsdomäne; Stop ist ein Wartezustand, nicht die Aussage, dass der M4 nie aus Reset gekommen sei. Der Cortex-M7, kurz M7, richtet gemeinsame Systemtakte ein und gibt die Freigabe über eine Hardwaresemaphore, abgekürzt HSEM. Der konkrete generierte Code und dessen Semaphorekennung sind maßgeblich.
Diese Beschreibung ist keine universelle Reset-Regel für jede Projektkonfiguration. Für den 35-Minuten-Versuch wird ein bereits funktionierendes Template bereitgestellt. Beide Images werden gebaut und geladen; die Startreihenfolge beziehungsweise die gemeinsame Debugkonfiguration wird vorab geprüft. Der M7 erhält keine zweite LED-Schleife. Eine Boot-Freigabe ist auch kein allgemeiner Schutz gemeinsamer Daten und stellt keine Cache-Kohärenz her; Cache bedeutet lokaler Zwischenspeicher. Solche Fragen behandeln spätere Vorlesungen. Heute bewahren wir den funktionierenden Start und ergänzen nur die kleine M4-Anwendung. Als Lehrzeitbasis des M4 verwenden wir 1000 HAL-Ticks pro Sekunde, also eine Millisekunde pro Tick. HAL bedeutet Hardware Abstraction Layer, Hardware-Abstraktionsschicht; Tick ist ein Zeitgeberereignis. SysTick ist der im Kern integrierte Zeitgeber, den das vorbereitete Projekt dafür nutzen kann. Ein Betriebssystem-Tick ist heute nicht aktiv. Die tatsächliche HAL-Konfiguration wird geprüft.

**Folie** 40 / 50

## CubeMX konfigurieren, in VS Code entwickeln




**Notizen** 40 / 50

STM32CubeMX ist das Konfigurationswerkzeug: Es speichert Board-, Pin-, Takt- und Peripherieeinstellungen in einer `.ioc`-Datei und erzeugt Start- und Initialisierungscode. VS Code, ausgeschrieben Visual Studio Code, ist der verwendete Editor. Die STM32 VS Code Extension ergänzt Projekt-, Build- und Debugfunktionen; ihre konkrete installierte Version und Bezeichnung werden im Versuch protokolliert. CMake erzeugt die Baukonfiguration für den Compiler. CM4 und CM7 bezeichnen die getrennten Firmwareziele für die zwei Kerne, keine austauschbaren Namen derselben Datei.
Die Pfeile zeigen den Entwicklungsweg von oben nach unten. ST-LINK ist der Debugadapter des Boards, der die übersetzten Images lädt und den Programmlauf beobachtet. Eine verfügbare CMake-Generator- und Dual-Core-Unterstützung muss zur installierten CubeMX-/Erweiterungsversion passen. Wir erfinden deshalb keine universelle Menüfolge. Im vorbereiteten Projekt sind beide Ziele bereits eingerichtet. Die Änderung einer Pinbelegung erfolgt in CubeMX, die Anwendungslogik in eigenen C-Dateien. Nach erneutem Generieren müssen diese Dateien weiterhin im tatsächlich verwendeten Ziel enthalten sein; ein erfolgreicher Edit ist noch kein erfolgreicher Build.

**Folie** 41 / 50

## Generierter Code und eigene Anwendung

- Initialisierung in generierten Dateien erhalten
- Eigene Funktionen unter `code/lab01/`
- Aufrufe in vorhandene `USER CODE`-Bereiche einfügen
- Keine zweite Endlosschleife in die vorhandene Schleife setzen
- Nach erneutem Generieren Einbindung und Build prüfen

**Notizen** 41 / 50

Generierte Dateien sind Ergebnis der CubeMX-Konfiguration und können bei erneuter Generierung ersetzt

**Notizen, Fortsetzung** 41 / 50

werden. Die markierten Kommentare `USER CODE BEGIN` und `USER CODE END` kennzeichnen dafür vorgesehene Erhaltungsbereiche. USER CODE bedeutet Anwendercode. Diese Kommentare sind eine Regel des Generators, keine Laufzeit- oder Hardwarefunktion. Die Erhaltung muss in den Generatoroptionen eingeschaltet sein; externe Versionsverwaltung bleibt zusätzlich sinnvoll.
Die vollständigen eigenen Anwendungsfunktionen liegen in `code/lab01/lab01_demo.c` und ihrem Header `lab01_demo.h`. Der Header beschreibt die aufrufbare Schnittstelle. Ein Include, also eine Header-Einbindung, und wenige Aufrufe werden in den passenden Anwenderbereichen der `main.c` des Cortex-M4, kurz M4, ergänzt. Die vorhandene `while (1)`-Schleife ruft einen einzelnen Anwendungsschritt auf. Eine zusätzliche Endlosschleife innerhalb ihres Anwenderblocks würde den übrigen generierten Kontrollfluss abschneiden. Auch der vorhandene Timer-Callback wird genau einmal ergänzt und nicht doppelt definiert. Nach einer Konfigurationsänderung prüfen Sie beide Build-Ziele, also die tatsächlich übersetzten Firmwarevarianten. Die Integration ist im zugehörigen README vollständig beschrieben; ein fertiges gerätespezifisches Cube-Projekt wird dadurch nicht behauptet.

**Folie** 42 / 50

## LED-Umschaltung ist nicht der volle Blinkzyklus

Ideales Lehrmodell: Toggle alle $500$ ms

- Toggle: aus wird an, an wird aus
- Zwei Toggles ergeben einen vollständigen Zyklus
- Zyklusdauer $= 1000$ ms, Blinkfrequenz $= 1$ Hz
- `HAL_Delay(500)` wartet im Hauptprogramm

Reale Abstände enthalten Aufrufdauer, Tickauflösung und weitere Arbeit.

**Notizen** 42 / 50

LED steht für Light-Emitting Diode, Leuchtdiode. Toggle bedeutet Umschalten des logischen Ausgangszustands. Bei einem anfänglich ausgeschalteten Ausgang macht die erste Umschaltung ihn an und die nächste wieder aus. Daher ergeben zwei Flankenabstände von je 500 Millisekunden einen vollständigen An-Aus-Zyklus von 1000 Millisekunden beziehungsweise einer Sekunde. Die Frequenz eines vollständigen Zyklus ist der Kehrwert seiner Dauer, hier ein Hertz. Die Flankenrate ist dagegen zwei pro Sekunde.
HAL heißt Hardware Abstraction Layer, Hardware-Abstraktionsschicht. `HAL_Delay(500)` wartet auf die HAL-Zeitbasis und gibt der Hauptschleife in dieser Zeit keine nächste Anwendungsarbeit. Interrupts können dabei weiterhin laufen, sofern sie zulässig sind. Der echte Abstand ist nicht automatisch exakt 500 Millisekunden: Tickauflösung, die Implementierung der Wartefunktion und zusätzlicher Code wirken mit. Das Lehrmodell erklärt die Zählung von Flanken und Zyklen, keine gemessene Frequenz. Eine im Hintergrund hinzugefügte Last verlängert den Abstand weiter. Im Labor unterscheiden wir daher eine Baseline, also einen Vergleichslauf ohne zusätzliche Last, von einer absichtlich belasteten Schleife.

**Folie** 43 / 50

### Hintergrundlast hat keine bekannte Millisekundenzahl

Vier bereitgestellte Versuchsmodi:

| Modus | Hauptschleife | LED-Auslöser |
|---|---|---|
| Baseline | Delay ohne Zusatzlast | Main |
| Polling konstant | feste Zählarbeit und Delay | Main |
| Polling variabel | wechselnde Zählarbeit und Delay | Main |
| Timer | wechselnde Zählarbeit | TIM2-Callback |

**Notizen** 43 / 50

Main ist die Hauptschleife der Anwendung. TIM2 ist Timerinstanz 2. LED bedeutet Light-Emitting Diode, Leuchtdiode. Der Callback ist die von der Hardwarebibliothek aufgerufene Anwendungsfunktion. Die Baseline liefert einen Vergleich ohne zusätzliche Rechenlast. Der Programmvergleich bildet nur eine periodische Ausgangsaktion nach; die Sensorprüfung bleibt das Papiermodell. In den beiden Polling-Modi erfolgt die Umschaltung in Main; die Schleife wartet danach 500 Millisekunden und enthält gegebenenfalls zusätzliche Arbeit. Im Timer-Modus schaltet ausschließlich der Callback die LED, während Main nur die variable Last bearbeitet.
Die Zählarbeit liest und verändert eine lokale `volatile`-Variable, sodass die erforderlichen Zugriffe nicht vollständig wegoptimiert werden. Eine festgelegte Iterationszahl ist trotzdem keine bekannte Millisekundendauer: Compiler, Optimierung und Hardware beeinflussen sie. Bei konstanter Arbeit kann ein im Modell konstanter zusätzlicher Versatz entstehen. Periodenjitter ist erst die Schwankung mehrerer Abstände, nicht schon deren vergrößerter Mittelwert. Die variable Last wechselt ihre Iterationszahl, um einen Unterschied untersuchen zu können. Die genaue Dauer muss gemessen werden. Jeder Modus wird nach einem Reset neu gestartet; ein Wechsel während eines laufenden Versuchs ist im bereitgestellten Modul nicht vorgesehen.

**Folie** 44 / 50

### Der Timer zählt unabhängig von der Schleifenrunde

Timer-Updatefrequenz:

$$f_{\mathrm{UPD}} = \frac{f_{\mathrm{TIM}}}{(PSC+1)(ARR+1)}$$

Nur bei tatsächlichem $f_{\mathrm{TIM}} = 100\,\mathrm{MHz}$:
$PSC = 9999$, $ARR = 4999$ → Update $2\,\mathrm{Hz}$ → LED-Zyklus $1\,\mathrm{Hz}$

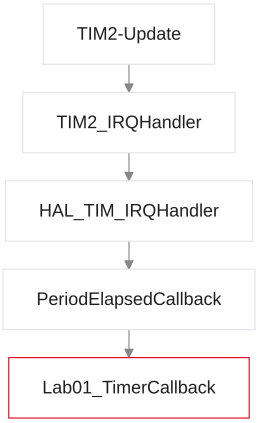
**Notizen** 44 / 50

Ein Hardwaretimer zählt mit einem eigenen zugeführten Takt. $f_{\text{TIM}}$ ist dieser tatsächliche Timer-Kerntakt in Hertz, nicht automatisch der CPU-Takt. CPU heißt Central Processing Unit, der Prozessorkern. $PSC$ ist der Prescaler-Wert, also der Vorteilerregisterwert; $ARR$ ist der Auto-Reload-Registerwert, also das Zählende. Für den verwendeten aufwärtszählenden Timer ergibt sich ein Update nach $(PSC+1)(ARR+1)$ Timer-Takten. $f_{\text{UPD}}$ ist die Frequenz dieser Updateereignisse.
Bei der ausdrücklich angenommenen Timerfrequenz von 100 Millionen Hertz teilen $9999 + 1 = 10000$ und $4999 + 1 = 5000$ zusammen durch 50 Millionen. Das ergibt zwei Updates pro Sekunde und einen Updateabstand von 500 Millisekunden. Schaltet der Callback bei jedem Update um, benötigt der vollständige LED-Zyklus zwei Updates und hat ein Hertz; LED ist die Leuchtdiode. Der echte Timer-Takt wird in CubeMX unter Beachtung des Bus-Vorteilers und der Timer-Taktauswahl geprüft. SystemCoreClock, die Variable für den Kerntakt, ersetzt diese Prüfung nicht. Andere Timer-Takte erfordern andere Registerwerte. Lange Handler oder gesperrte Unterbrechungen können die Softwarebedienung trotz regelmäßiger Hardwareupdates verzögern oder mehrere Ereignisse zusammenfallen lassen.

**Folie** 45 / 50

## Timer und Aufrufkette

Aufrufkette im vorbereiteten M4-Projekt




**Notizen** 45 / 50

TIM2 bezeichnet Timerinstanz 2; ein Update ist hier das periodische Zählereignis. `TIM2_IRQHandler` ist der generierte Interrupt-Handler. IRQ bedeutet Interrupt Request, die Anforderung. `HAL_TIM_IRQHandler` ist die Timer-Behandlung der Hardware Abstraction Layer, der Hardware-Abstraktionsschicht. Sie prüft und behandelt die Timerquelle nach den Bibliotheksregeln und ruft bei einem Update `HAL_TIM_PeriodElapsedCallback` auf. Im Diagramm ist dessen Name auf „PeriodElapsedCallback“ verkürzt. Der vorhandene Callback delegiert einmal an `Lab01_TimerCallback`, unsere Anwendungsfunktion.
Die Pfeile sind Aufrufe beziehungsweise Weiterleitung, keine garantierten Laufzeiten. Jede Stufe läuft hier auf dem Cortex-M4, kurz M4, im Handlerkontext. Der Callback ist daher kein neuer Task und läuft nicht parallel zur Hauptschleife. Der Timer muss zuvor initialisiert und im Interruptmodus gestartet sein, seine Anforderung im Nested Vectored Interrupt Controller, kurz NVIC, zugelassen sein, und das richtige M4-Image muss den Handler enthalten. Ein fehlendes Glied verhindert die Umschaltung. Der Code setzt

**Notizen, Fortsetzung** 45 / 50

nicht zusätzlich Peripherieflags an zufälliger Stelle zurück. Diese saubere Delegation erlaubt, die kleine eigene Reaktion getrennt von generiertem Startcode zu lesen.

**Folie** 46 / 50

### Der kleine Timer-Callback

Aus `code/lab01/lab01_demo.c` · vollständige Integration im README

```
void Lab01_TimerCallback(TIM_HandleTypeDef *htim)
{
    if (initialized && mode == LAB01_TIMER &&
        htim != NULL && htim->Instance == TIM2) {
        ++lab01_diag.timer_callbacks;
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    }
}
```

**Notizen** 46 / 50

Der C-Callback hat keinen Rückgabewert. `TIM_HandleTypeDef` ist der Timer-Handle-Typ der HAL, Hardware Abstraction Layer. Ein Handle ist eine Verwaltungsstruktur für eine Peripherieinstanz. `htim` ist ein Zeiger auf diese Struktur; `NULL` bezeichnet einen ungültigen leeren Zeiger. Nach der Nullprüfung liest `htim->Instance` die Timerinstanz. TIM2 bezeichnet Timer 2. Die Prüfung ist nötig, weil der übergeordnete Callback auch andere Timer bedienen kann.
`initialized` zeigt, dass die einmalige Anwendungsinitialisierung erfolgreich war. `mode` enthält den gewählten Versuchsmodus; `LAB01_TIMER` bezeichnet den Timer-Versuch. Die mit `&&` verknüpften Bedingungen werden in C von links nach rechts mit Kurzschluss ausgewertet, sodass ein Nullzeiger nicht dereferenziert wird. `lab01_diag.timer_callbacks` zählt beobachtete Callback-Aufrufe, nicht alle physisch erzeugten Hardwareereignisse. `++` erhöht ihn um eins. `HAL_GPIO_TogglePin` schaltet den General-Purpose Input/Output, kurz GPIO, also digitalen Ausgang PE1 um: GPIOE ist Port E, GPIO_PIN_1 dessen Pin 1. Dort liegt die gelbe LED, Light-Emitting Diode beziehungsweise Leuchtdiode. Der Callback enthält weder Delay noch Hintergrundlast. Nach der Initialisierung ändert Main die geprüften Steuerdaten nicht mehr; Diagnosedaten werden für einen konsistenten Stand erst im angehaltenen Lauf gelesen.

**Folie** 47 / 50

## Ein fairer Vergleich hält Bedingungen fest

| Vergleich | Beibehalten | Bewusst verändern |
|---|---|---|
| Baseline / Polling konstant | Build, Takte, LED-Pin | zusätzliche Last |
| Konstant / variabel | Auslöser Main | Iterationsfolge |
| Polling / Timer | Zielabstand, Build, Board | Auslöseort |

Vor jedem Lauf Modus wählen, beide Images laden und neu starten.

**Notizen** 47 / 50

Ein Vergleich ist nur interpretierbar, wenn die bewusste Änderung und die übrigen Bedingungen dokumentiert sind. Baseline bedeutet den Vergleichslauf ohne Zusatzlast. Main ist die Hauptschleife. LED ist die Light-Emitting Diode, Leuchtdiode. Build bezeichnet hier den konkreten Übersetzungslauf einschließlich Optimierungsoptionen. Die Polling-Modi, also die aktiv abfragende beziehungsweise zyklisch arbeitende Hauptschleife, sollen denselben nominalen Flankenabstand wie der Timer-Modus haben: 500 Millisekunden.
Dieser Zielabstand ist keine Behauptung identischer realer Abstände. Die Hauptschleife wartet zusätzlich auf ihre Arbeit; der Timer liefert Updates aus seinem Takt. Der Wechsel auf den Timer verändert auch die Auslöseart und den Unterbrechungsaufwand. Mit dieser Unterscheidung untersuchen wir genau die Architekturwirkung. Die erstmalige Einrichtung des Projekts ist vorbereitet; im Zeitblock wird nur die Modusauswahl geändert und neu gebaut. Vor jedem Lauf werden die zwei Kern-Images geladen und gemeinsam korrekt gestartet. Debug-Haltephasen gehören nicht in eine Zeitmessreihe. Ohne Messgerät darf die LED qualitativ beobachtet werden, aber eine optische Einschätzung liefert keine Mikrosekundenwerte und keinen Jitternachweis.

**Folie** 48 / 50

## Ein Messprotokoll benennt seine Grenzen

- Beginn und Ende: etwa zwei aufeinanderfolgende Ausgangsflanken
- Einheit, Trigger, Auflösung und Anzahl der Abstände
- Modus, Build, Takte und andere Lasten
- Minimum, Maximum, Mittelwert und Spanne erst nach Messung
- Beobachtete Periodenspanne ist keine WCET-Schranke

**Notizen** 48 / 50

Für den Grundversuch ist die messbare Größe der Abstand zweier aufeinanderfolgender Ausgangsumschaltungen. Alternierende steigende und fallende Flanken liefern den Toggle-Abstand; nur steigende Flanken liefern den vollständigen Blinkzyklus. Der Trigger ist die Bedingung, mit der das Oszilloskop

**Notizen, Fortsetzung** 48 / 50

die Aufzeichnung auslöst. Auflösung beschreibt, wie fein die Zeitpunkte unterschieden werden können. Anzahl der Abstände, Modus, Übersetzungsoptionen, tatsächliche Takte und sonstige Lasten gehören zum Ergebnis.
Die Peak-to-Peak-Spanne ist Maximum minus Minimum der gewählten Größe. Sie ist mit dieser Größe und ihrer Einheit zu beschriften. WCET heißt Worst-Case Execution Time und bezeichnet den maximalen eigenen Ausführungsbedarf im angenommenen Betriebsmodell; eine beobachtete Periodenspanne ist etwas anderes. Ein Puls um Hintergrundarbeit enthält mögliche Unterbrechungen und Markeraufwand. Eine reine Interrupt-Latenz bis zur ersten Handlerinstruktion wird im Grundversuch nicht gemessen. Für sie wären ein zugeordnetes Eingangsevent und ein passender Endpunkt nötig. Die Protokolltabelle bleibt leer, bis ein realer Versuch erfolgt. Eine Hostprüfung untersucht Code auf dem Entwicklungsrechner, nicht auf dem Mikrocontroller. Qualitatives Blinken, solche Hostprüfungen des C-Codes und Hardware-Zeitmessungen werden als unterschiedliche Prüfarten berichtet.

**Folie** 49 / 50

## Transfer: eine schnelle Erfassung genügt nicht

Lehrfall mit $D = 3$ ms:

- Handler erfasst den Auftrag früh
- Hauptschleife beginnt die Bearbeitung erst nach $3$ ms
- Bearbeitung braucht weitere $1$ ms

Ist die vollständige Antwort rechtzeitig? Welche Grenze fehlt im Entwurf?

**Notizen** 49 / 50

Die Antwort lautet nein. $D$ ist die relative Deadline, also die zulässige Dauer von der Freigabe bis zum vollständigen Abschluss. Der Handler erfasst den Auftrag zwar früh, aber die geforderte Bearbeitung beginnt erst drei Millisekunden nach der Freigabe und dauert eine weitere Millisekunde. Ohne weitere Unterbrechungen ergibt sich die Antwortzeit $R = 3 + 1 = 4$ Millisekunden. Die Frist wird um eine Millisekunde überschritten.
Im Foreground/Background-Muster, also der Teilung zwischen kurzer Vordergrund-Unterbrechungsarbeit und längerer Hauptschleife, fehlt hier eine ausreichend kleine Grenze für das Warten im Hintergrund. „Kurzer Handler“ beantwortet nur eine Teilfrage. Mögliche Verbesserungen sind eine Zerlegung langer Hintergrundschritte oder eine andere Zuteilung der Arbeit; jede benötigt erneut eine passende Gesamtbilanz. Ein zusätzlicher Fehler wäre ein verlorener Auftrag durch ein ungeeignetes Flag, also ein einzelnes Merkmal. Dessen Datenvertrag muss ebenso stimmen. Die Aufgabe verbindet damit Funktion, vollständige Zeitkette und Programmstruktur. Genau diese drei Punkte werden in der Übung und im Quiz noch einmal geprüft, bevor die nächste Vorlesung mehrere konkurrierende Aufträge modelliert.

**Folie** 50 / 50

## Mit einem Zeitvertrag in Vorlesung 2

- Funktion und Frist gemeinsam fordern
- Eigene Rechnung von vollständiger Antwort trennen
- Polling und Interrupts unter Annahmen beurteilen
- Beobachtung und Garantie auseinanderhalten
- Nächster Schritt: mehreren Jobs Prozessorzeit zuteilen

Quellen: Buttazzo · Kopetz · ST RM0399, PM0214, UM2408

**Notizen** 50 / 50

Die wichtigste Verbindung des Termins lautet: Ein Ergebnis braucht den richtigen Wert und einen zulässigen Abschlusszeitpunkt. Dazu gehören ein klarer Beginn, eine relative Frist, der definierte Abschluss und die Lastannahmen. Die eigene Rechenzeit ist nur ein Anteil der Antwortzeit. Polling kann bei begrenzten Runden passen; Interrupts können bestimmte Reaktionen vorziehen, ohne automatisch Ereignisvollständigkeit oder eine Gesamtfrist zu garantieren. Ein tatsächlich beobachteter Erfolg ist enger als eine allgemeine Garantie.

Vorlesung 2 verwendet diese Grundlage für Scheduling, die Zuteilung von Prozessorzeit an mehrere Jobs, also einzelne Aufgabenausführungen. Bewahren Sie das vorbereitete Projekt, die CubeMX-Konfigurationsdatei und das ausgefüllte Protokoll auf. CubeMX ist das Konfigurations- und Generierungswerkzeug. Theoriequelle ist Giorgio C. Buttazzo, Hard Real-Time Computing Systems; ergänzend Hermann Kopetz, Real-Time Systems. STMicroelectronics liefert RM0399, das Reference Manual beziehungsweise Peripheriehandbuch des STM32H755, PM0214, das Programming Manual des Cortex-M4, und UM2408, das User Manual des Nucleo-Boards. Versionsabhängige Werkzeug- und Bootdetails werden am tatsächlich verwendeten Projekt geprüft. Die Unterlagen enthalten Lehrzahlen, Anwendungsbausteine und offene Messfelder, keine behaupteten Boardläufe.

Zur Nachbereitung: Kursliteratur, STM32CubeMX, STM32 VS Code Extension, RM0399, PM0214, UM2408. Die Dokumentkennungen benennen Referenzen; eine ungelesene aktuelle Dokumentrevision gilt nicht als nachgeprüfte Werkzeugkonfiguration.